

Notes on Chapter 23: Hierarchical Data

Norman Lim

2025-08-14

Main idea: This chapter is about **rectangling** – the art of converting hierarchical data (or tree-like data) into rectangular data

Prerequisites

Installing the required libraries:

```
library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

library(repurrrsive)
library(jsonlite)

##
## Attaching package: 'jsonlite'
##
## The following object is masked from 'package:purrr':
##
##     flatten
```

Lists

Creating lists with `list()`:

```
x1 <- list(1:4, "a", TRUE)

x1

## [[1]]
## [1] 1 2 3 4
##
## [[2]]
## [1] "a"
##
## [[3]]
```

```
## [1] TRUE
```

Creating a list with named components or “children”:

```
x2 <- list(a = 1:2, b = 1:3, c = 1:4)
```

```
x2
```

```
## $a
## [1] 1 2
##
## $b
## [1] 1 2 3
##
## $c
## [1] 1 2 3 4
```

Displaying the structure of a list with `str()`:

```
str(x1)
```

```
## List of 3
## $ : int [1:4] 1 2 3 4
## $ : chr "a"
## $ : logi TRUE
```

```
str(x2)
```

```
## List of 3
## $ a: int [1:2] 1 2
## $ b: int [1:3] 1 2 3
## $ c: int [1:4] 1 2 3 4
```

Note: this takes up less space than printing the contents of a list. The result looks a bit like the output of `glimpse()` for dataframes.

Hierarchy

Creating a nested list (a list within a list):

```
x3 <- list(list(1, 2), list(3, 4))
str(x3)
```

```
## List of 2
## $ :List of 2
## ..$ : num 1
## ..$ : num 2
## $ :List of 2
## ..$ : num 3
## ..$ : num 4
```

The concatenator `c()`, on the other hand, creates a flat vector:

```
c(c(1, 2), c(3, 4))
```

```
## [1] 1 2 3 4
```

```
x4 <- c(list(1, 2), list(3, 4))
str(x4)
```

```
## List of 4
```

```
## $ : num 1
## $ : num 2
## $ : num 3
## $ : num 4
```

Example of an extremely-nested list:

```
x5 <- list(1, list(2, list(3, list(4, list(5)))))
str(x5)
```

```
## List of 2
## $ : num 1
## $ :List of 2
## ..$ : num 2
## ..$ :List of 2
## .. ..$ : num 3
## .. ..$ :List of 2
## .. .. ..$ : num 4
## .. .. ..$ :List of 1
## .. .. .. ..$ : num 5
```

As the list gets more complex, it becomes more difficult to see its structure, even when `str()` is used. The command `View()` is more practical when working with complex lists. This command will show the contents of a list in a gui.

List-columns

Lists that live inside a tibble are called list-columns.

Example of a list-column:

```
df <- tibble(
  x = 1:2,
  y = c("a", "b"),
  z = list(list(1, 2), list(3, 4, 5))
)
df
```

```
## # A tibble: 2 x 3
##       x y      z
##   <int> <chr> <list>
## 1     1 a    <list [2]>
## 2     2 b    <list [3]>
```

Using `dplyr` functions in a tibble with list-columns:

```
df |>
  filter(x == 1)
```

```
## # A tibble: 1 x 3
##       x y      z
##   <int> <chr> <list>
## 1     1 a    <list [2]>
```

The list-columns behave like any other column, but they require more compute resources. The process of unnesting can help mitigate this drawback.

Unnesting

Basic forms of list-columns:

- **named:** The names are usually consistent for every row.
- **unnamed:** The number of elements tends to vary for each row.

Example of a named list-column:

```
df1 <- tribble(
  ~x, ~y,
  1, list(a = 11, b = 12),
  2, list(a = 21, b = 22),
  3, list(a = 31, b = 32)
)
df1
```

```
## # A tibble: 3 x 2
##       x y
##   <dbl> <list>
## 1     1 <named list [2]>
## 2     2 <named list [2]>
## 3     3 <named list [2]>
```

Example of an unnamed list-column:

```
df2 <- tribble(
  ~x, ~y,
  1, list(11, 12, 13),
  2, list(21),
  3, list(31, 32)
)
df2
```

```
## # A tibble: 3 x 2
##       x y
##   <dbl> <list>
## 1     1 <list [3]>
## 2     2 <list [1]>
## 3     3 <list [2]>
```

For these two cases, the functions `unnest_wider()` and `unnest_longer()` are commonly used.

unnest_wider()

This is commonly used when each row has the same number of elements, like in the case of a named list-column:

```
df1 |>
  unnest_wider(y)
```

```
## # A tibble: 3 x 3
##       x     a     b
##   <dbl> <dbl> <dbl>
## 1     1    11    12
## 2     2    21    22
## 3     3    31    32
```

The names of the new columns were, by default, obtained from the names of the list elements.

Changing the default name for the new columns generated by `unnest_wider()` using the `names_sep` argument:

```
df1 |>
  unnest_wider(y, names_sep = "_")
```

```
## # A tibble: 3 x 3
##       x   y_a y_b
##   <dbl> <dbl> <dbl>
## 1     1    11    12
## 2     2    21    22
## 3     3    31    32
```

`unnest_longer()`

Unnesting a tibble with an unnamed list-column using `unnest_longer()`:

```
df2 |>
  unnest_longer(y)
```

```
## # A tibble: 6 x 2
##       x     y
##   <dbl> <dbl>
## 1     1    11
## 2     1    12
## 3     1    13
## 4     2    21
## 5     3    31
## 6     3    32
```

In this case, the values for `x` column gets duplicated for each element inside `y`. In case one of the elements is empty, we get:

```
df6 <- tribble(
  ~x, ~y,
  "a", list(1, 2),
  "b", list(3),
  "c", list()
)

df6 |> unnest_longer(y)
```

```
## # A tibble: 3 x 2
##       x     y
##   <chr> <dbl>
## 1 a         1
## 2 a         2
## 3 b         3
```

With nothing to “distribute” `x` with, the row disappears. If you want to preserve the row with missing list, the argument `keep_empty = TRUE` can be used:

```
df6 |> unnest_longer(y, keep_empty = TRUE)
```

```
## # A tibble: 4 x 2
##       x     y
##   <chr> <dbl>
## 1 a         1
## 2 a         2
## 3 b         3
```

```
## 4 c      NA
```

Inconsistent types

Unnesting a data with a list-column that contains different types of vector:

```
df4 <- tribble(  
  ~x, ~y,  
  "a", list(1),  
  "b", list("a", TRUE, 5)  
)  
  
df4 |> unnest_longer(y)
```

```
## # A tibble: 4 x 2  
##   x     y  
##   <chr> <list>  
## 1 a     <dbl [1]>  
## 2 b     <chr [1]>  
## 3 b     <lgl [1]>  
## 4 b     <dbl [1]>
```

In this case, `unnest_longer()` cannot fully unnest the list-column because it can't find a common type of vector. The original list-column will only be unnested until single-element lists remain, in order for the new column to have a consistent vector type – a list, in this case.

Other functions

- `unnest_auto()`: automatically picks between `unnest_longer()` and `unnest_wider()` depending on the structure of the list-column. It might be convenient to use initially but it can “spoil” the user because it doesn't force the user to understand how the data is structured.
- `unnest()`: expands both rows and columns. Useful when the list-column contains a 2d structure.

Case studies

Let's begin by creating a very wide data:

```
repos <- tibble(json = gh_repos)  
repos
```

```
## # A tibble: 6 x 1  
##   json  
##   <list>  
## 1 <list [30]>  
## 2 <list [30]>  
## 3 <list [30]>  
## 4 <list [26]>  
## 5 <list [30]>  
## 6 <list [30]>
```

The tibble contains 6 rows, but each row contains an unnamed list with either 26 or 30 elements.

Since these lists are unnamed, it can be a good idea to start with `unnest_longer()` to deal with the data:

```
repos |>  
  unnest_longer(json)
```

```
## # A tibble: 176 x 1
##   json
##   <list>
## 1 <named list [68]>
## 2 <named list [68]>
## 3 <named list [68]>
## 4 <named list [68]>
## 5 <named list [68]>
## 6 <named list [68]>
## 7 <named list [68]>
## 8 <named list [68]>
## 9 <named list [68]>
## 10 <named list [68]>
## # i 166 more rows
```

Now each element is named so it is now possible to use `unnest_wider()`:

```
repos |>
  unnest_longer(json) |>
  unnest_wider(json)

## # A tibble: 176 x 68
##       id name full_name owner private html_url description fork url
##       <int> <chr> <chr>    <list>    <lg1> <chr>    <chr>    <lg1> <chr>
## 1  6.12e7 after gaborcsa~ <named list> FALSE https://~ Run Code i~ FALSE http~
## 2  4.05e7 argu~ gaborcsa~ <named list> FALSE https://~ Declarativ~ FALSE http~
## 3  3.64e7 ask gaborcsa~ <named list> FALSE https://~ Friendly C~ FALSE http~
## 4  3.49e7 base~ gaborcsa~ <named list> FALSE https://~ Do we get ~ FALSE http~
## 5  6.16e7 cite~ gaborcsa~ <named list> FALSE https://~ Test R pac~ TRUE http~
## 6  3.39e7 clis~ gaborcsa~ <named list> FALSE https://~ Unicode sy~ FALSE http~
## 7  3.72e7 cmak~ gaborcsa~ <named list> FALSE https://~ port of cm~ TRUE http~
## 8  6.80e7 cmark gaborcsa~ <named list> FALSE https://~ CommonMark~ TRUE http~
## 9  6.32e7 cond~ gaborcsa~ <named list> FALSE https://~ <NA> TRUE http~
## 10 2.43e7 cray~ gaborcsa~ <named list> FALSE https://~ R package ~ FALSE http~
## # i 166 more rows
## # i 59 more variables: forks_url <chr>, keys_url <chr>,
## # collaborators_url <chr>, teams_url <chr>, hooks_url <chr>,
## # issue_events_url <chr>, events_url <chr>, assignees_url <chr>,
## # branches_url <chr>, tags_url <chr>, blobs_url <chr>, git_tags_url <chr>,
## # git_refs_url <chr>, trees_url <chr>, statuses_url <chr>,
## # languages_url <chr>, stargazers_url <chr>, contributors_url <chr>, ...
```

The operation above works! However, the resulting data has so many columns that it is still difficult to see even the column names.

Displaying the first 10 column names:

```
repos |>
  unnest_longer(json) |>
  unnest_wider(json) |>
  names() |>
  head(10)

## [1] "id" "name" "full_name" "owner" "private"
## [6] "html_url" "description" "fork" "url" "forks_url"
```

Selecting only the relevant columns (or the “interesting” ones):

```

repos |>
  unnest_longer(json) |>
  unnest_wider(json) |>
  select(id, full_name, owner, description)

## # A tibble: 176 x 4
##       id full_name      owner      description
##   <int> <chr>          <list>      <chr>
## 1 61160198 gaborcsardi/after  <named list [17]> Run Code in the Background
## 2 40500181 gaborcsardi/argufy  <named list [17]> Declarative function argu~
## 3 36442442 gaborcsardi/ask     <named list [17]> Friendly CLI interaction ~
## 4 34924886 gaborcsardi/baseimports <named list [17]> Do we get warnings for un~
## 5 61620661 gaborcsardi/citest  <named list [17]> Test R package and repo f~
## 6 33907457 gaborcsardi/clisymbols <named list [17]> Unicode symbols for CLI a~
## 7 37236467 gaborcsardi/cmaker  <named list [17]> port of cmake to r
## 8 67959624 gaborcsardi/cmark   <named list [17]> CommonMark parsing and re~
## 9 63152619 gaborcsardi/conditions <named list [17]> <NA>
## 10 24343686 gaborcsardi/crayon  <named list [17]> R package for colored ter~
## # i 166 more rows

```

Now it is a bit easier to make sense of the data. Each child in the hierarchy is a GitHub user and the list for each user contains the GitHub repositories that they created.

If we try expanding or unnesting the `owner` list-column, we would get an error message:

```

repos |>
  unnest_longer(json) |>
  unnest_wider(json) |>
  select(id, full_name, owner, description) |>
  unnest_wider(owner)

#> Error in `unnest_wider()` :
#> ! Can't duplicate names between the affected columns and the original
#>   data.
#> x These names are duplicated:
#>   i `id`, from `owner`.
#> i Use `names_sep` to disambiguate using the column name.
#> i Or use `names_repair` to specify a repair strategy.

```

This is because the list-column `owner` also contains an `id` column and we already have a column with this name. This is where the `names_sep` argument comes in handy:

```

repos |>
  unnest_longer(json) |>
  unnest_wider(json) |>
  select(id, full_name, owner, description) |>
  unnest_wider(owner, names_sep = "_")

## # A tibble: 176 x 20
##       id full_name      owner_login owner_id owner_avatar_url owner_gravatar_id
##   <int> <chr>          <chr>      <int> <chr>          <chr>
## 1 61160198 gaborcsardi~ gaborcsardi 660288 https://avatars~ ""
## 2 40500181 gaborcsardi~ gaborcsardi 660288 https://avatars~ ""
## 3 36442442 gaborcsardi~ gaborcsardi 660288 https://avatars~ ""
## 4 34924886 gaborcsardi~ gaborcsardi 660288 https://avatars~ ""
## 5 61620661 gaborcsardi~ gaborcsardi 660288 https://avatars~ ""
## 6 33907457 gaborcsardi~ gaborcsardi 660288 https://avatars~ ""

```



```
## 7 37236467 gaborcsardi~ gaborcsardi 660288 https://avatars~ ""
## 8 67959624 gaborcsardi~ gaborcsardi 660288 https://avatars~ ""
## 9 63152619 gaborcsardi~ gaborcsardi 660288 https://avatars~ ""
## 10 24343686 gaborcsardi~ gaborcsardi 660288 https://avatars~ ""
## # i 166 more rows
## # i 14 more variables: owner_url <chr>, owner_html_url <chr>,
## #   owner_followers_url <chr>, owner_following_url <chr>,
## #   owner_gists_url <chr>, owner_starred_url <chr>,
## #   owner_subscriptions_url <chr>, owner_organizations_url <chr>,
## #   owner_repos_url <chr>, owner_events_url <chr>,
## #   owner_received_events_url <chr>, owner_type <chr>, ...
```

Relational data

Nested data is sometimes used to represent that is usually spread across multiple data frames.

```
chars <- tibble(json = got_chars)
chars
```

```
## # A tibble: 30 x 1
##   json
##   <list>
## 1 <named list [18]>
## 2 <named list [18]>
## 3 <named list [18]>
## 4 <named list [18]>
## 5 <named list [18]>
## 6 <named list [18]>
## 7 <named list [18]>
## 8 <named list [18]>
## 9 <named list [18]>
## 10 <named list [18]>
## # i 20 more rows
```

Again, this is has an unnamed list-column so it is a good idea to start with `unnest_wider()`:

```
chars |>
  unnest_wider(json)
```

```
## # A tibble: 30 x 18
##   url          id name  gender culture born  died  alive titles aliases father
##   <chr>      <int> <chr> <chr>   <chr>   <chr> <chr> <lgl> <list> <list> <chr>
## 1 https://w~ 1022 Theo~ Male   "Ironb~ "In ~ ""    TRUE  <chr> <chr>  ""
## 2 https://w~ 1052 Tyri~ Male   ""      "In ~ ""    TRUE  <chr> <chr>  ""
## 3 https://w~ 1074 Vict~ Male   "Ironb~ "In ~ ""    TRUE  <chr> <chr>  ""
## 4 https://w~ 1109 Will  Male   ""      ""      "In ~ FALSE <chr> <chr>  ""
## 5 https://w~ 1166 Areo~ Male   "Norvo~ "In ~ ""    TRUE  <chr> <chr>  ""
## 6 https://w~ 1267 Chett Male   ""      "At ~ "In ~ FALSE <chr> <chr>  ""
## 7 https://w~ 1295 Cres~ Male   ""      "In ~ "In ~ FALSE <chr> <chr>  ""
## 8 https://w~ 130 Aria~ Female "Dorni~ "In ~ ""    TRUE  <chr> <chr>  ""
## 9 https://w~ 1303 Daen~ Female "Valyr~ "In ~ ""    TRUE  <chr> <chr>  ""
## 10 https://w~ 1319 Davo~ Male   "Weste~ "In ~ ""    TRUE  <chr> <chr>  ""
## # i 20 more rows
## # i 7 more variables: mother <chr>, spouse <chr>, allegiances <list>,
## #   books <list>, povBooks <list>, tvSeries <list>, playedBy <list>
```

Selecting “interesting” or relevant columns, to make the result easier to read:

```
characters <- chars |>
  unnest_wider(json) |>
  select(id, name, gender, culture, born, died, alive)
```

```
characters
```

```
## # A tibble: 30 x 7
##       id name                gender culture    born                died alive
##   <int> <chr>                <chr>  <chr>    <chr>          <chr> <lgl>
## 1  1022 Theon Greyjoy      Male   "Ironborn" "In 278 AC or 279 AC,~ ""    TRUE
## 2  1052 Tyrion Lannister  Male   ""         "In 273 AC, at Caster~ ""    TRUE
## 3  1074 Victarion Greyjoy Male   "Ironborn" "In 268 AC or before,~ ""    TRUE
## 4  1109 Will              Male   ""         ""         "In ~ FALSE
## 5  1166 Areo Hotah        Male   "Norvoshi" "In 257 AC or before,~ ""    TRUE
## 6  1267 Chett             Male   ""         "At Hag's Mire"      "In ~ FALSE
## 7  1295 Cressen           Male   ""         "In 219 AC or 220 AC" "In ~ FALSE
## 8   130 Arianne Martell   Female "Dornish"  "In 276 AC, at Sunspe~ ""    TRUE
## 9  1303 Daenerys Targaryen Female "Valyrian" "In 284 AC, at Dragon~ ""    TRUE
## 10 1319 Davos Seaworth    Male   "Westeros" "In 260 AC or before,~ ""    TRUE
## # i 20 more rows
```

The columns chosen were not list-columns. Where are the list-columns? To find them, the functions `where()` and the `is.list` can be used:

```
chars |>
  unnest_wider(json) |>
  select(id, where(is.list))
```

```
## # A tibble: 30 x 8
##       id titles      aliases    allegiances books      povBooks tvSeries playedBy
##   <int> <list>    <list>    <list>    <list>    <list>    <list>    <list>
## 1  1022 <chr [2]> <chr [4]> <chr [1]> <chr [3]> <chr [2]> <chr [6]> <chr>
## 2  1052 <chr [2]> <chr [11]> <chr [1]> <chr [2]> <chr [4]> <chr [6]> <chr>
## 3  1074 <chr [2]> <chr [1]> <chr [1]> <chr [3]> <chr [2]> <chr [1]> <chr>
## 4  1109 <chr [1]> <chr [1]> <NULL>    <chr [1]> <chr [1]> <chr [1]> <chr>
## 5  1166 <chr [1]> <chr [1]> <chr [1]> <chr [3]> <chr [2]> <chr [2]> <chr>
## 6  1267 <chr [1]> <chr [1]> <NULL>    <chr [2]> <chr [1]> <chr [1]> <chr>
## 7  1295 <chr [1]> <chr [1]> <NULL>    <chr [2]> <chr [1]> <chr [1]> <chr>
## 8   130 <chr [1]> <chr [1]> <chr [1]> <chr [4]> <chr [1]> <chr [1]> <chr>
## 9  1303 <chr [5]> <chr [11]> <chr [1]> <chr [1]> <chr [4]> <chr [6]> <chr>
## 10 1319 <chr [4]> <chr [5]> <chr [2]> <chr [1]> <chr [3]> <chr [5]> <chr>
## # i 20 more rows
```

Exploring the titles column, which we can now see as an unnamed list-column:

```
chars |>
  unnest_wider(json) |>
  select(id, titles) |>
  unnest_longer(titles)
```

```
## # A tibble: 59 x 2
##       id titles
##   <int> <chr>
## 1  1022 "Prince of Winterfell"
## 2  1022 "Lord of the Iron Islands (by law of the green lands)"
## 3  1052 "Acting Hand of the King (former)"
```

```
## 4 1052 "Master of Coin (former)"
## 5 1074 "Lord Captain of the Iron Fleet"
## 6 1074 "Master of the Iron Victory"
## 7 1109 ""
## 8 1166 "Captain of the Guard at Sunspear"
## 9 1267 ""
## 10 1295 "Maester"
## # i 49 more rows
```

The result is a table. To flatten the data, we can use joins to merge the tables nested in each list-column. But first we need to clean them:

```
titles <- chars |>
  unnest_wider(json) |>
  select(id, titles) |>
  unnest_longer(titles) |>
  filter(titles != "") |>
  rename(title = titles)
```

```
titles
```

```
## # A tibble: 52 x 2
##       id title
##   <int> <chr>
## 1  1022 Prince of Winterfell
## 2  1022 Lord of the Iron Islands (by law of the green lands)
## 3  1052 Acting Hand of the King (former)
## 4  1052 Master of Coin (former)
## 5  1074 Lord Captain of the Iron Fleet
## 6  1074 Master of the Iron Victory
## 7  1166 Captain of the Guard at Sunspear
## 8  1295 Maester
## 9   130 Princess of Dorne
## 10 1303 Queen of the Andals and the Rhoynar and the First Men, Lord of the Sev~
## # i 42 more rows
```

We can then proceed to create a table like this for each list-column in the data and then using joins to combine them with the `character` data to produce flattened version of the original.

Deeply nested

In this case, repeated unnesting is necessary.

```
gmaps_cities
```

```
## # A tibble: 5 x 2
##   city      json
##   <chr>    <list>
## 1 Houston  <named list [2]>
## 2 Washington <named list [2]>
## 3 New York <named list [2]>
## 4 Chicago  <named list [2]>
## 5 Arlington <named list [2]>
```

The data looks initially “harmless” – just a two-column dataset, with the second column being a 2-element list-column. Unnesting, we get:

```
gmaps_cities |>
  unnest_wider(json)
```

```
## # A tibble: 5 x 3
##   city      results    status
##   <chr>    <list>    <chr>
## 1 Houston <list [1]> OK
## 2 Washington <list [2]> OK
## 3 New York <list [1]> OK
## 4 Chicago <list [1]> OK
## 5 Arlington <list [2]> OK
```

To make things simpler in this example, we will drop the `status` column and continue to explore the remaining list-column:

```
gmaps_cities |>
  unnest_wider(json) |>
  select(-status) |>
  unnest_longer(results)
```

```
## # A tibble: 7 x 2
##   city      results
##   <chr>    <list>
## 1 Houston <named list [5]>
## 2 Washington <named list [5]>
## 3 Washington <named list [5]>
## 4 New York <named list [5]>
## 5 Chicago <named list [5]>
## 6 Arlington <named list [5]>
## 7 Arlington <named list [5]>
```

The result, again, is a named list, so we can use `unnest_wider()`:

```
locations <- gmaps_cities |>
  unnest_wider(json) |>
  select(-status) |>
  unnest_longer(results) |>
  unnest_wider(results)
```

```
locations
```

```
## # A tibble: 7 x 6
##   city      address_components formatted_address geometry place_id types
##   <chr>    <list>                <chr>          <list>    <chr>  <list>
## 1 Houston <list [4]>                Houston, TX, USA <named list> ChIJAYW~ <list>
## 2 Washington <list [2]>                Washington, USA <named list> ChIJ-bD~ <list>
## 3 Washington <list [4]>                Washington, DC, USA <named list> ChIJW-T~ <list>
## 4 New York <list [3]>                New York, NY, USA <named list> ChIJ0wg~ <list>
## 5 Chicago <list [4]>                Chicago, IL, USA <named list> ChIJ7cv~ <list>
## 6 Arlington <list [4]>                Arlington, TX, USA <named list> ChIJ05g~ <list>
## 7 Arlington <list [4]>                Arlington, VA, USA <named list> ChIJD6e~ <list>
```

Exploring the exact location of cities with similar names:

```
locations |>
  select(city, formatted_address, geometry) |>
  unnest_wider(geometry)
```

```
## # A tibble: 7 x 6
##   city      formatted_address bounds      location      location_type viewport
##   <chr>    <chr>                <list>      <list>        <chr>        <list>
## 1 Houston Houston, TX, USA <named list> <named list> APPROXIMATE <named list>
## 2 Washin~ Washington, USA <named list> <named list> APPROXIMATE <named list>
## 3 Washin~ Washington, DC, ~ <named list> <named list> APPROXIMATE <named list>
## 4 New Yo~ New York, NY, USA <named list> <named list> APPROXIMATE <named list>
## 5 Chicago Chicago, IL, USA <named list> <named list> APPROXIMATE <named list>
## 6 Arling~ Arlington, TX, U~ <named list> <named list> APPROXIMATE <named list>
## 7 Arling~ Arlington, VA, U~ <named list> <named list> APPROXIMATE <named list>
```

Unnesting the list-column location will give us the coordinates:

```
locations |>
  select(city, formatted_address, geometry) |>
  unnest_wider(geometry) |>
  unnest_wider(location)
```

```
## # A tibble: 7 x 7
##   city      formatted_address bounds      lat      lng location_type viewport
##   <chr>    <chr>                <list>    <dbl>    <dbl> <chr>        <list>
## 1 Houston Houston, TX, USA <named list> 29.8   -95.4 APPROXIMATE <named list>
## 2 Washin~ Washington, USA <named list> 47.8  -121. APPROXIMATE <named list>
## 3 Washin~ Washington, DC, ~ <named list> 38.9  -77.0 APPROXIMATE <named list>
## 4 New Yo~ New York, NY, USA <named list> 40.7  -74.0 APPROXIMATE <named list>
## 5 Chicago Chicago, IL, USA <named list> 41.9  -87.6 APPROXIMATE <named list>
## 6 Arling~ Arlington, TX, U~ <named list> 32.7  -97.1 APPROXIMATE <named list>
## 7 Arling~ Arlington, VA, U~ <named list> 38.9  -77.1 APPROXIMATE <named list>
```

Extracting the bounds:

```
locations |>
  select(city, formatted_address, geometry) |>
  unnest_wider(geometry) |>
  select(!location:viewport) |>
  unnest_wider(bounds)
```

```
## # A tibble: 7 x 4
##   city      formatted_address northeast      southwest
##   <chr>    <chr>                <list>        <list>
## 1 Houston Houston, TX, USA <named list [2]> <named list [2]>
## 2 Washington Washington, USA <named list [2]> <named list [2]>
## 3 Washington Washington, DC, USA <named list [2]> <named list [2]>
## 4 New York New York, NY, USA <named list [2]> <named list [2]>
## 5 Chicago Chicago, IL, USA <named list [2]> <named list [2]>
## 6 Arlington Arlington, TX, USA <named list [2]> <named list [2]>
## 7 Arlington Arlington, VA, USA <named list [2]> <named list [2]>
```

Renaming the southwest and northeast corners of the rectangular bounds to make it shorter and more intuitive:

```
locations |>
  select(city, formatted_address, geometry) |>
  unnest_wider(geometry) |>
  select(!location:viewport) |>
  unnest_wider(bounds) |>
  rename(ne = northeast, sw = southwest) |>
```

```
unnest_wider(c(ne, sw), names_sep = "_")
```

```
## # A tibble: 7 x 6
##   city      formatted_address ne_lat ne_lng sw_lat sw_lng
##   <chr>      <chr>          <dbl> <dbl> <dbl> <dbl>
## 1 Houston    Houston, TX, USA      30.1  -95.0   29.5  -95.8
## 2 Washington Washington, USA       49.0 -117.   45.5 -125.
## 3 Washington Washington, DC, USA   39.0  -76.9   38.8 -77.1
## 4 New York   New York, NY, USA    40.9  -73.7   40.5 -74.3
## 5 Chicago    Chicago, IL, USA     42.0  -87.5   41.6 -87.9
## 6 Arlington  Arlington, TX, USA   32.8  -97.0   32.6 -97.2
## 7 Arlington  Arlington, VA, USA   38.9  -77.0   38.8 -77.2
```

Using the function `hoist()` to extract directly the deeply-nested elements that we are interested in (similar results to the multiple unnesting above):

```
locations |>
  select(city, formatted_address, geometry) |>
  hoist(
    geometry,
    ne_lat = c("bounds", "northeast", "lat"),
    sw_lat = c("bounds", "southwest", "lat"),
    ne_lng = c("bounds", "northeast", "lng"),
    sw_lng = c("bounds", "northeast", "lng")
  )
```

```
## # A tibble: 7 x 7
##   city      formatted_address ne_lat sw_lat ne_lng sw_lng geometry
##   <chr>      <chr>          <dbl> <dbl> <dbl> <dbl> <list>
## 1 Houston    Houston, TX, USA      30.1   29.5  -95.0  -95.0 <named list [4]>
## 2 Washington Washington, USA       49.0   45.5 -117.  -117. <named list [4]>
## 3 Washington Washington, DC, USA   39.0   38.8  -76.9  -76.9 <named list [4]>
## 4 New York   New York, NY, USA    40.9   40.5  -73.7  -73.7 <named list [4]>
## 5 Chicago    Chicago, IL, USA     42.0   41.6  -87.5  -87.5 <named list [4]>
## 6 Arlington  Arlington, TX, USA   32.8   32.6  -97.0  -97.0 <named list [4]>
## 7 Arlington  Arlington, VA, USA   38.9   38.8  -77.0  -77.0 <named list [4]>
```

This is neat way of dealing with deeply-nested data, but we can only use this after understanding the structure of our dataset. If a data dictionary is provided, we might be able to use this with less data exploration.

JSON

This file structure is designed to be easily read (and written) by machines. Generally has six data types:

- the null type: like `NA` in R.
- the string type: always wrapped in double quotes.
- the number type: can be integers, decimals, and can be in scientific notation, but does not support `Inf` or `NaN` values.
- the boolean type: logical conditions in lower case (`true` and `false`).

An **unnamed** list is called an **array** in JSON.

A **named** list is called an **object**.

jsonlite

Using the `jsonlite` package to convert JSON file into one with R data structure:

```

# a path to a json file inside the jsonlite package
gh_users_json()

## [1] "/home/norman/R/x86_64-pc-linux-gnu-library/4.5/repurrrsive/extdata/gh_users.json"
# reading
gh_users2 <- read_json(gh_users_json())

# Checking
identical(gh_users, gh_users2)

## [1] TRUE

Parsing strings with json structure:
str(parse_json('1'))

## int 1
str(parse_json('[1, 2, 3]'))

## List of 3
## $ : int 1
## $ : int 2
## $ : int 3
str(parse_json('{"x": [1, 2, 3]}'))

## List of 1
## $ x:List of 3
## ..$ : int 1
## ..$ : int 2
## ..$ : int 3

```

Starting the rectangling process

Creating a sample tibble with JSON structure:

```

json <- '[
  {"name": "John", "age": 34},
  {"name": "Susan", "age": 27}
]'

df <- tibble(json = parse_json(json))
df

```

```

## # A tibble: 2 x 1
##   json
##   <list>
## 1 <named list [2]>
## 2 <named list [2]>

```

Unnesting the sample data:

```

df |>
  unnest_wider(json)

## # A tibble: 2 x 2
##   name    age
##   <chr> <int>

```

```
## 1 John      34
## 2 Susan     27
```

A JSON file with a single top-level JSON object:

```
json <- '{
  "status": "OK",
  "results": [
    {"name": "John", "age": 34},
    {"name": "Susan", "age": 27}
  ]
}'
```

To unnest this, we need to first wrap it in a list before we put it in a tibble:

```
df <- tibble(json = list(parse_json(json)))
df
```

```
## # A tibble: 1 x 1
##   json
##   <list>
## 1 <named list [2]>
```

Unnesting:

```
df |>
  unnest_wider(json) |>
  unnest_longer(results) |>
  unnest_wider(results)
```

```
## # A tibble: 2 x 3
##   status name    age
##   <chr>  <chr> <int>
## 1 OK    John    34
## 2 OK    Susan   27
```

An alternative approach is to reach inside the parsed JSON and start with the part that we are interested in:

```
df <- tibble(results = parse_json(json)$results)
df |>
  unnest_wider(results)
```

```
## # A tibble: 2 x 2
##   name    age
##   <chr> <int>
## 1 John    34
## 2 Susan   27
```